

Step 1 — Mini SRS: *AI Chatbot API Wrapper*

Project name: AI Chatbot API Wrapper

Stakeholders: Developer team, Security lead, DevOps engineer, End users (clients/apps), Product owner

1. Purpose

Provide a secure, lightweight REST API wrapper around the OpenAI Chat API that applications can call to send user prompts and receive model responses. The wrapper centralizes usage, enforces rate limits, abstracts model details, and — critically — protects the OpenAI API key(s) and any client tokens from leakage.

2. Scope

- Expose endpoints for: `/v1/chat` (send prompt & get response), `/v1/conversations` (list/persist conversations), basic CRUD on saved prompts, and admin endpoints to rotate keys.
- Authentication for clients (API tokens or OAuth2 bearer tokens).
- Server-side secure storage of OpenAI API key(s).
- Logging, metrics, and basic rate-limiting.
- Deployment using Docker and Kubernetes (minikube for lab).
- Focus: ensure secrets (OpenAI API key, client tokens) are never exposed in logs, responses, or container images.

3. Functional Requirements (FR)

FR1 — **Chat endpoint:** POST `/v1/chat` accepts `{client_id, conversation_id, message}` and returns AI reply.

FR2 — **Authentication:** All client API calls must present a bearer token. Unauthorized requests return 401.

FR3 — **Conversation CRUD:** Create/read/update/delete conversation metadata (no sensitive storage of raw API keys).

FR4 — **Admin key management:** Admin can rotate OpenAI API key via protected admin endpoint or via Kubernetes Secrets.

FR5 — **Rate limiting:** Per-client rate limiting configurable (default 60 requests/min).

FR6 — **Audit logging:** Log request metadata (`client_id`, endpoint) but never log `Authorization` header, full prompt content if flagged private, or the OpenAI API key.

FR7 — **Health & metrics:** `/health` and `/metrics` endpoints for k8s health checks and Prometheus scraping.

4. Non-Functional Requirements (NFR)

NFR1 — **Security:** Secrets must be stored encrypted at rest (e.g., via K8s Secrets + encryption) and in-memory for minimal time. No API keys in source repo or image.

NFR2 — **Availability:** 99.9% uptime target for production. App should gracefully handle transient OpenAI API failures with exponential backoff.

NFR3 — **Performance:** Typical response latency < 1.5s (excluding OpenAI model latency) under normal load.

NFR4 — **Scalability:** App must run horizontally in Kubernetes.

NFR5 — **Observability:** Structured logs (JSON), traces, and metrics must be emitted.

NFR6 — **Maintainability:** Codebase modular (routes, auth, key-manager, services).

5. Constraints

- Must use Flask (Python) for the wrapper.
- OpenAI API key use is limited by the plan; rate-limiting must protect the shared key.
- In lab: use minikube/local Kubernetes; prefer secrets mounted via k8s Secret.
- Local dev may use a `.env` file but production images must not contain secrets.
- No third-party secret manager configured in the lab beyond Kubernetes Secrets (optional placeholder for HashiCorp Vault).

6. Acceptance Criteria (concrete, testable)

AC1 — Unauthenticated request to `/v1/chat` returns 401 Unauthorized.

AC2 — Authenticated request with valid client token returns a 200 with `{"reply": "..."};` the OpenAI API key is never present in logs or response body.

AC3 — When admin rotates the OpenAI key (via the admin endpoint or by updating the Kubernetes Secret), subsequent chat calls use the new key without requiring container rebuild.

AC4 — Container image build logs and image contents do not include the OpenAI API key (verified by scanning image layers).

AC5 — Rate limiting rejects excessive requests with 429 and logs the event (without logging tokens).

AC6 — Health endpoint returns 200 when app is functional and 503 when critical dependency (e.g., keys missing) is absent.

7. Minimal Data Model / Storage

- `clients` collection/table: {`client_id`, `hashed_token`, `rate_limit_config`, `scopes`, `created_at`}
- `conversations` collection/table: {`conversation_id`, `client_id`, `summary`, `created_at`, `updated_at`}
- **Secrets store:** OpenAI API key stored only in Kubernetes Secret or environment variable passed at runtime and never checked into repo.

8. Security Considerations (quick list)

- Use HTTPS/TLS for all client-app traffic.
- Store client tokens hashed (e.g., HMAC or bcrypt) and validate via secure comparison.
- Rotate OpenAI key regularly; support hot rotation.
- Redact and avoid storing PII in logs.

- Use minimal privileges for Kubernetes service accounts.
- Use CSP/HTTP security headers where relevant.